

1. LaTeX이란 무엇인가

이 문서는 Overleaf나 VS Code 같은 특정 도구와 무관하게, **LaTeX 자체가 무엇이고 어떻게 동작하는지**를 설명합니다. 이 저장소의 어떤 템플릿을 쓰든(학위논문, 학술지 투고, 포스터, 발표) 공통으로 알아 두면 좋은 내용입니다. 실제로 논문을 쓰기 시작하려면 [2-OVERLEAF_THESIS_PROJECT.md](#) 또는 [3-VSCODE_TeXLive_SETUP.md](#)를 참고하세요.

LaTeX은 무엇이고, 왜 쓰는가

LaTeX(레이텍)은 문서의 **내용**과 **형식**을 분리해서 다루는 조판(組版) 시스템입니다. 한글이나 워드처럼 화면에 보이는 대로 편집하는 프로그램(WYSIWYG, What You See Is What You Get)이 아니라, .tex라는 일반 텍스트 파일에 내용과 명령을 함께 적어 두면 컴파일러가 이를 읽어 PDF로 조판해 줍니다.

	워드프로세서(한글, MS Word)	LaTeX
편집 방식	화면에 보이는 대로 직접 배치	텍스트로 작성 → 컴파일해서 결과 확인
표·그림·참고문헌 번호	手동으로 관리하다 순서가 바뀌면 직접 수정	<code>\label/\ref</code> 로 자동 관리, 순서가 바뀌어도 자동 갱신
여러 사람과의 협업	파일을 주고받으며 버전이 꼬이기 쉬움	텍스트 파일이라 git으로 변경 이력 관리가 쉬움
수식	별도 수식 편집기 필요	문법으로 직접 작성(x^2 등), 정렬·번호 자동
대용량 문서	스크롤이 느려지고 스타일이 깨지기 쉬움	장별 파일로 나눠 관리해도 스타일이 일관됨

학위논문처럼 표·그림·수식·참고문헌이 많고, 장이 여러 개이며, 형식(여백·글꼴·번호 체계)을 학교 규정에 정확히 맞춰야 하는 문서에서 LaTeX의 장점이 특히 두드러집니다. 반대로 짧고 서식이 단순한 문서, 또는 여러 사람이 실시간으로 같은 화면을 보며 편집해야 하는 경우에는 워드프로세서가 더 편할 수 있습니다.

기본 동작 원리: 소스 → 컴파일 → PDF

LaTeX 작업의 기본 흐름은 다음과 같습니다.

1. .tex 파일(텍스트)에 내용과 명령을 작성한다.
2. **컴파일러**(엔진)가 .tex 파일을 읽어 PDF를 만든다.
3. 결과 PDF를 확인하고, 필요하면 소스를 고쳐 다시 컴파일한다.

이 저장소의 템플릿은 이 반복을 `KNUE_thesis_main_build.cmd/.sh` 같은 빌드 스크립트로 자동화해 둔 것입니다.

왜 한 번이 아니라 여러 번 컴파일하는가

표·그림·참고문헌 번호, 목차, 상호 참조(\ref)는 **컴파일을 한 번 해 봐야 몇 번인지 알 수 있는 값**입니다. 그래서 LaTeX은 보통 다음 순서로 여러 번 실행됩니다.

1. 1차 컴파일: 문서를 훑으며 각 표·그림·수식·장 번호를 계산해 .aux 파일에 기록한다. 이 시점에는 아직 최종 번호를 모르므로 참조 자리에 물음표(??)가 남을 수 있다.
2. 참고문헌 처리(biber 또는 bibtex): .bib 파일과 .aux를 대조해 참고문헌 목록을 만든다.
3. 2차(이후) 컴파일: 앞서 기록된 .aux/참고문헌 정보를 읽어 모든 번호와 참조를 실제 값으로 채운다.

이 저장소의 빌드 스크립트(latexmk 기반)는 이 반복 횟수를 자동으로 판단해서 필요한 만큼 다시 실행해 줍니다. 그래서 보통은 "몇 번 컴파일해야 하는지" 직접 셀 필요가 없습니다.

컴파일할 때 만들어지는 부산물 파일

컴파일 과정에서 .aux, .log, .bbl, .bcf, .out, .toc, .fls, .fdb_latexmk 같은 파일이 함께 생깁니다. 이들은 컴파일러가 참조 번호·참고문헌·목차를 기억해 두는 임시 파일이며, **원고가 아니므로 직접 편집하지 않습니다**. 이 저장소의 빌드 스크립트는 이런 부산물을 프로젝트 폴더 밖의 캐시 폴더에서 만들고 완성된 PDF만 복사해 오도록 되어 있으며(자세한 내용은 [manual/8-KNUE_THESIS_TEMPLATE_USAGE.md](#) 참고), clean 명령으로 언제든지 지우고 다시 만들 수 있습니다.

기본 문법

LaTeX 문서는 크게 세 가지 요소로 이루어집니다.

명령(command)

\ 뒤에 이름이 오고, 필요하면 중괄호 {}로 인수를 받습니다.

```
\section{서론}           % 인수 1개
\includegraphics{그림.pdf} % 파일명을 인수로 받음
\textbf{굵게 표시할 글자} % 굵게
```

대괄호 []는 **선택 인수**(생략 가능한 옵션)를 나타냅니다.

```
\includegraphics[width=0.8\linewidth]{그림.pdf}
```

환경(environment)

\begin{이름}과 \end{이름}으로 감싸는 구간입니다. 여러 줄에 걸친 구조(표, 그림, 목록, 수식 블록 등)를 나타낼 때 씁니다.

```
\begin{itemize}
  \item 첫 번째 항목
  \item 두 번째 항목
\end{itemize}
```

패키지(package)

LaTeX 기본 기능만으로는 부족한 표·그림·색상·수식 기능을 추가해 주는 확장 모듈입니다. 문서 맨 앞(preamble)에서 \usepackage{}로 불러옵니다. 이 템플릿에서는 sub/0-preamble.tex에 필요한 패키지가 이미 정리되어 있으므로, 새 기능이 필요할 때만 추가하면 됩니다.

```
\usepackage{graphicx} % 그림 삽입
\usepackage{amsmath}  % 수식 확장
```

주석(comment)

% 뒤의 내용은 컴파일 시 무시됩니다. 이 템플릿의 각 장 파일 위쪽에 있는 안내문도 이 방식으로 되어 있습니다.

% 이 줄은 PDF에 출력되지 않습니다.

문서의 기본 골격

모든 LaTeX 문서는 아래와 같은 최소 구조를 가집니다. 이 템플릿에서는 이 구조를 KNUE_thesis_main.tex(전체 조립)와 sub/ 폴더의 장별 파일들이 나누어 담당합니다.

```
\documentclass{어떤 문서 종류인지} % 예: article, report, 또는 이 템플릿의 KNUE_thesis
```

```
% --- 여기(preamble)에서 패키지를 불러오고 공통 설정을 합니다 ---
```

```
\usepackage{graphicx}
```

```
\begin{document}
```

```
\section{제목}
```

```
본문 내용...
```

```
\end{document}
```

\begin{document} 앞부분을 **preamble(전문, 前文)**이라 부르며, 문서 전체에 적용되는 설정(패키지, 여백, 글꼴, 사용자 정의 명령)을 이곳에 모아 둡니다. 이 템플릿에서는 sub/0-preamble.tex가 이 역할을 합니다.

표·그림·수식·참고문헌: 번호는 직접 쓰지 않는다

LaTeX에서 가장 자주 헛갈리는 부분이자, 가장 큰 장점이기도 한 규칙입니다. “표 1”, “그림 3-2”처럼 번호를 **직접 타이핑하지 않고**, 아래 세 명령으로 관리합니다.

- \caption{} — 표/그림의 제목. 이 위치에서 번호가 자동으로 매겨집니다.
- \label{} — 나중에 이 표/그림을 다시 부를 때 쓸 이름표(식별자)를 붙입니다.
- \ref{} — 본문에서 그 번호를 불러옵니다.

```
\begin{table}[htbp]
```

```
  \caption{집단별 평균 점수}
```

```
  \label{tab:example}
```

```
  ...
```

```
\end{table}
```

표~\ref{tab:example}에서 볼 수 있듯이 ...

이렇게 해 두면 앞쪽에 표가 하나 추가되어 번호가 밀리더라도, \ref{tab:example}이 가리키는 번호가 **자동으로 갱신**됩니다. 표·그림뿐 아니라 수식, 장·절 번호, 참고문헌 인용도 같은 원리로 동작합니다. 이 템플릿에서 실제로 표·그림·수식·참고문헌을 작성하는 구체적인 명령(\ThesisTableStyle, \figf, 인용 명령 등)은 [manual/8-KNUE_THESIS_TEMPLATE_USAGE.md](#)에 정리되어 있습니다.

컴파일 엔진: pdfLaTeX, XeLaTeX, LuaLaTeX

LaTeX 문서를 PDF로 만드는 프로그램을 **엔진**이라 부르며, 세 가지가 널리 쓰입니다. 한글처럼 유니코드·시스템 글꼴이 필요한 문서에서는 어떤 엔진을 쓰는지 결과가 큰 영향을 줍니다.

엔진	주요 특징	이 저장소 템플릿에서의 권장 여부
pdfLaTeX	가장 오래된 표준 엔진. 빠르지만 유니코드·한글·시스템 글꼴 처리에 불리하다.	권장하지 않음 — 한글 글꼴이 제대로 나오지 않거나 오류가 날 수 있음

엔진	주요 특징	이 저장소 템플릿에서의 권장 여부
XeLaTeX	유니코드와 시스템 글꼴을 비교적 잘 처리해 한글 문서에도 쓸 수 있다.	가능은 하지만 권장하지 않음 — 줄바꿈·글꼴·표지 위치가 LuaLaTeX 기준인 이 템플릿과 미묘하게 달라질 수 있음
LuaLaTeX	유니코드, 시스템 글꼴, 최신 패키지와의 호환성이 가장 좋다.	권장 — 이 저장소의 모든 템플릿과 빌드 스크립트의 기준 엔진

이 저장소의 템플릿은 모두 **LuaLaTeX 기준**으로 표지 간격·한글 글꼴·줄바꿈을 맞췄습니다. Overleaf에서 컴파일러를 pdfLaTeX이나 XeLaTeX로 설정하면 `\directlua` 같은 LuaLaTeX 전용 코드가 PDF에 그대로 보이거나 컴파일이 실패할 수 있으므로, 반드시 **Compiler: LuaLaTeX**로 맞춰야 합니다. 로컬 빌드 스크립트(`KNUE_thesis_main_build.cmd/.sh` 등)는 이미 LuaLaTeX을 쓰도록 설정되어 있어 별도로 신경 쓸 필요가 없습니다.

자주 보는 용어 정리

용어	의미
컴파일(compile) 엔진(engine)	.tex 소스를 읽어 PDF를 만드는 과정 컴파일을 수행하는 프로그램(pdfLaTeX/XeLaTeX/LuaLaTeX)
preamble(전문)	<code>\begin{document}</code> 이전의, 문서 전체 설정을 모아 두는 부분
패키지(package) 환경(environment)	<code>\usepackage{}</code> 로 불러오는 기능 확장 모듈 <code>\begin{...}\end{}</code> 으로 감싸는 구조(표, 그림, 목록 등)
float .aux/.log/.bbl 등	표·그림처럼 본문 사이를 “떠다니며” 배치되는 요소 컴파일 중 생기는 임시 부산물 파일. 원고가 아니므로 편집하지 않음
<code>\label/\ref</code>	번호를 자동으로 매기고 불러오는 상호 참조 명령

더 배우고 싶다면

- Overleaf의 LaTeX 입문 문서: <https://www.overleaf.com/learn>
- 한글 LaTeX 커뮤니티 자료, TeX Stack Exchange(질의응답): <https://tex.stackexchange.com/>

이 저장소의 템플릿을 실제로 쓰기 시작하려면 [2-OVERLEAF_THESIS_PROJECT.md](#)(웹에서 바로 시작) 또는 [3-VSCODE_TeXLive_SETUP.md](#)(내 컴퓨터에 설치)로 넘어가세요.